Java Object-Oriented Programming Assignment 2
Library Management System

Muğla Sıtkı Koçman University Software Engineering

May 14, 2025

1 Objective

The objective of this assignment is to practice object-oriented programming in Java by implementing a library management system using core OOP principles including inheritance, interfaces, abstract classes, and method overloading.

2 Problem Statement

Implement a Java program that models a library management system. The system should include the following classes and concepts:

- **User (Abstract Class)**: A base abstract class with attributes for ID, name, email, password, and borrowed books. Contains abstract methods such as `displayInfo()`.
- **Member**: Inherits from `User`. Has functionality to borrow and return books. Demonstrate method overloading with the `borrowBook()` method (e.g., by ID or by object).
- **Librarian**: Inherits from `User` and implements the `BookManageable` interface. Has functionality to manage the book inventory.
- **Book**: Represents a book with an ID, title, and author.
- **BookManageable (Interface)**: Declares methods for `addBook(Book)` and `removeBook(Book)` to be implemented by the `Librarian` class.

3 Requirements

1. Implement the classes with appropriate attributes and methods.
2. Use encapsulation: all attributes should be private with public getters and setters.
3. Use appropriate access modifiers and OOP design practices.
4. Use arrays or lists where necessary to manage books and borrowed books.
5. Demonstrate method overloading in the `Member` class with multiple versions of the `borrowBook()` method. One of the borrow books by their ID and the other borrow books by Book object.
6. Abstract methods in `User` must be implemented in both `Member` and `Librarian`.
7. `Librarian` must implement the `BookManageable` interface and its methods.
8. Test your implementation thoroughly in the `main` method.

4 Expected Output for Main.java

```
User{id=1, name='Alice␣Brown', email='alice@example.com', password='member123'}
BorrowedBooks{books=}
User{id=2, name='Bob␣White', email='bob@example.com', password='member456'}
BorrowedBooks{books=}
User{id=3, name='Mr.␣Green', email='green@example.com', password='lib789'}
Library Inventory:
Book{id=1, title='The␣Great␣Gatsby', author='F.␣Scott␣Fitzgerald'}
Book{id=2, title='1984', author='George␣Orwell'}

After borrowing books:
User{id=1, name='Alice␣Brown', email='alice@example.com', password='member123'}
BorrowedBooks{books=Book{id=1, title='The␣Great␣Gatsby', author='F.␣Scott␣Fitzgerald'
    }}
User{id=2, name='Bob␣White', email='bob@example.com', password='member456'}
BorrowedBooks{books=Book{id=2, title='1984', author='George␣Orwell'}}
User{id=3, name='Mr.␣Green', email='green@example.com', password='lib789'}
Library Inventory:
(no books available)

After returning a book:
User{id=1, name='Alice␣Brown', email='alice@example.com', password='member123'}
BorrowedBooks{books=}
Library Inventory:
Book{id=1, title='The␣Great␣Gatsby', author='F.␣Scott␣Fitzgerald'}
```

5 Class Design Guidelines

- **User (abstract)**
  Fields: `id, name, email, password, borrowedBooks[]`
  Abstract Method: `displayInfo()`

- **Member**
  Inherits `User`
  Methods: `borrowBook(Book)`, `borrowBook(int bookId)`, `returnBook(Book)`

- **Librarian**
  Inherits `User`, Implements `BookManageable`
  Methods: `addBook(Book)`, `removeBook(Book)`

- **BookManageable (interface)**
  Methods: `addBook(Book)`, `removeBook(Book)`

6 Submission Guidelines

- Submit a ZIP file containing all the Java source files.
- Ensure your code is well-commented and formatted.